

Operazioni su file e descrittori

FUNZIONE

int open (const char* pathname, int flags, mode_t mode);

int open (const char* pathname, int flags);

- **Argomenti e funzionamento**
 - La funzione restituisce un descrittore per il file specificato in **pathname**.
 - Il **secondo argomento flags** deve obbligatoriamente contenere una delle seguenti modalità di accesso:
 - **O_RDONLY** (sola lettura),
 - **O_WRONLY** (sola scrittura), o **O_RDWR** (lettura e scrittura).
 - Si possono specificare ulteriori flag (e.g., **O_APPEND**, **O_TRUNC**) concatenandoli tramite OR bit a bit.
 - Se **O_CREAT** è specificato, il file viene creato se non è già presente nel sistema
 - se **O_CREAT** ed **O_EXCL** sono entrambi specificati, viene restituito un errore se il file che si desidera creare è già esistente nel sistema.
 - I flag descritti sono definiti in `<fcntl.h>`. **Quando O_CREAT viene specificato**, è necessario fornire alla funzione tramite l'argomento **mode** i permessi di creazione per il file (se non già presente nel sistema).
 - **Per la specifica dei permessi si può utilizzare la codifica ottale** (e.g., 0660) oppure una delle macro definite in `<sys/stat.h>`.
- **Valori di ritorno**
 - In caso di **successo**, viene restituito il descrittore così aperto (un intero ≥ 0). Altrimenti,
 - **viene restituito -1** e la variabile **errno** indicherà la causa dell'errore (e.g., **EINTR**)

ssize_t read(int fd, void* buf, size_t count);

- **Argomenti e funzionamento**
 - La funzione tenta di leggere **count** bytes dal descrittore **fd** e di memorizzarli in **buf**.
- **Valori di ritorno**
 - In caso di successo, come valore di ritorno viene restituito il numero di byte letti, o 0 nel caso la fine del file sia stata già raggiunta. Si noti come sia possibile che il numero di byte letti sia minore del numero richiesto; questo può accadere per svariate ragioni, ad esempio perché leggendo quei byte si è raggiunta la fine del file, oppure perché al momento dell'operazione solo quel numero di byte era disponibile, o ancora perché l'arrivo di un segnale ha provocato un'interruzione nel mezzo dell'operazione.
 - In caso di errore, viene restituito -1 e la variabile **errno** indicherà la causa dell'errore (e.g., **EINTR**, **EBADF**, **EIO**). **CONTROLLARE LA POSSIBILE INTERRUZIONE CON `ERRNO == EINTR`**

ssize_t write(int fd, void* buf, size_t count);

- **Argomenti e funzionamento**
 - La funzione tenta di scrivere **count** bytes sul descrittore **fd** leggendoli da **buf**.
- **Valori di ritorno**
 - In caso di successo, come valore di ritorno viene restituito il numero di byte scritti effettivamente. Si noti come sia possibile che tale numero sia inferiore rispetto al numero richiesto; questo può accadere per svariate ragioni, ad esempio perché lo spazio sul mezzo fisico di destinazione è insufficiente, oppure perché l'arrivo di un segnale ha provocato un'interruzione nel mezzo dell'operazione.
 - In caso di errore, viene restituito -1 e la variabile **errno** indicherà la causa dell'errore (e.g., **EINTR**, **EBADF**, **EIO**). **CONTROLLARE LA POSSIBILE INTERRUZIONE CON `ERRNO == EINTR`**

int close(int fd);

- **Argomenti e funzionamento**
 - Chiude un descrittore di file, così che possa essere riutilizzato dal sistema. Se fd è l'ultimo descrittore aperto associato ad uno specifico file, le risorse associate all'apertura del file vengono liberate dal sistema operativo.
 - L'operazione di close() si applica a descrittori sia di file ordinari che di file speciali, quali socket e pipe.
- **Valori di ritorno**
 - **In caso di successo, viene restituito 0.**
 - **Altrimenti, viene restituito -1** e la variabile errno indicherà la causa dell'errore (e.g., EINTR).

int unlink(const char* pathname);

- **Argomenti e funzionamento**
 - **SERVE PER I PATH CONDIVISI** La funzione rimuove il nome pathname dal filesystem.
 - Se pathname è l'ultimo hardlink 1 esistente al file, il file viene candidato per la rimozione.
 - Se non vi sono descrittori aperti per quel file nei processi in esecuzione, il file viene rimosso immediatamente e le risorse ad esso associate liberate;
 - se invece vi è almeno un processo con un descrittore aperto sul file, la rimozione viene posticipata fino all'avvenuta chiusura dell'ultimo descrittore
- **Valori di ritorno**
 - **In caso di successo, viene restituito 0.**
 - **Altrimenti, viene restituito -1** e la variabile errno indicherà la causa dell'errore (e.g., EINTR)

ESEMPIO LETTURA E SCRITTURA SU FILE:

```
#define DEFAULT_BLOCK_SIZE 128;

static inline void performCopyBetweenDescriptors(int src_fd, int dest_fd, int block_size)
{
    char* buf = malloc(block_size);
    //LETTURA SU UN FD
    while (1) {
        //READ
        int read_bytes = 0; // index for writing into the buffer
        int bytes_left = block_size; // number of bytes to (possibly) read
        while (bytes_left > 0) {
            int ret = read(src_fd, buf+read_bytes, bytes_left); //nota che legge da src, e
            //scrive nel buffer nella giusta posizione
            if (ret==0) break;
            if (ret == -1) {
                if (errno == EINTR) continue; //interrotto prima della lettura di un byte
                else exit(EXIT_FAILURE); //interrotto per qualche errore
            }
            bytes_left-=ret;//può capitare di leggere meno bytes del block_size. allora
            //aggiorno le variabili e continuo
            read_bytes+=ret;//finchè non lo leggo tutto
        }
        if (read_bytes == 0) break;

        //SCITTURA SU UN FD
        int written_bytes = 0; // index for reading from the buffer
        bytes_left = read_bytes; // number of bytes to write

        while (bytes_left > 0) {
            int ret = write(dest_fd, buf+written_bytes, bytes_left);
            if (ret==-1){
                if(errno == EINTR) continue;
                else exit(EXIT_FAILURE);
            }
            bytes_left-=ret;
            written_bytes+=ret;
        }
    }

    free(buf);
}

int main(int argc, char* argv[]) {
    int block_size, src_fd, dest_fd;

    if (argc == 4) {
        block_size = atoi(argv[3]);
    } else {
        block_size = DEFAULT_BLOCK_SIZE;
    }
}
```

```

if (argc < 3 || argc > 4)
    handle_error("Syntax: <source_file> <dest_file> [<block_size>]\n");
if (block_size <= 0)
    handle_error("Blocksize must be positive");

// create descriptors for source and destination files
src_fd = open(argv[1], O_RDONLY);
if (src_fd < 0) handle_error("Could not open source file");

// for simplicity we use rw-r--r-- permissions for the destination file
dest_fd = open(argv[2], O_WRONLY | O_CREAT | O_EXCL, 0644);
if (dest_fd < 0){
    if(errno == EEXIST) {
        fprintf(stderr, "WARNING: file %s already exists, I will overwrite it!\n",
argv[2]);
        dest_fd = open(argv[2], O_WRONLY | O_CREAT, 0644);
    }else
        handle_error("Could not create destination file");
}

// use a helper method to actually perform the copy
performCopyBetweenDescriptors(src_fd, dest_fd, block_size);

// close the descriptors
int ret = close(src_fd);
if (ret < 0) handle_error("Could not close source file");
ret = close(dest_fd);
if (ret < 0) handle_error("Could not close destination file");

exit(EXIT_SUCCESS);
}

```

Esecuzione Corrente

FUNZIONI

2.1 Creazione di un processo figlio

`pid_t fork(void);`

- **Argomenti e funzionamento**
 - Questa funzione crea un nuovo processo duplicando il processo chiamante. Il nuovo processo è detto figlio, quello chiamante padre. Padre e figlio vengono eseguiti in spazi di memoria separati.
 - È necessario includere `<unistd.h>` per usarla.
- **Valore di ritorno:**
 - In caso di **successo**, la funzione restituisce **0** nel processo figlio ed il **PID del figlio** nel processo padre. In caso di **fallimento**, viene restituito **-1** ed `errno` viene impostata col codice dell'errore occorso.

2.2 Attesa terminazione di un processo figlio

`pid_t wait(int *status);`

- **Argomenti e funzionamento**
 - Questa funzione consente ad un processo padre di mettersi in attesa della terminazione dei suoi processi figli. Se **status** è diverso da **NULL**, al termine della chiamata esso conterrà delle informazioni sullo stato del processo figlio che è terminato.
 - È necessario includere `<sys/wait.h>` per usarla.
- **Valore di ritorno:**
 - In caso di **successo**, la funzione restituisce il **PID del processo figlio terminato**. In caso di **fallimento**, viene **restituito -1** ed `errno` viene impostata col codice dell'errore occorso.

2.3 Creazione di un thread

Nelle applicazioni multithread si include `<pthread.h>`, ed al momento della compilazione è necessario linkare l'eseguibile prodotto alla libreria `pthread`. Si tenga presente inoltre che in caso di fallimento le funzioni per la gestione dei thread non aggiornano la variabile `errno` per indicarne la causa, ma restituiscono il codice di errore come valore di ritorno!

`int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)(void*), void *arg);`

- **Argomenti e funzionamento**
 - Questa funzione viene usata per creare un nuovo thread, con attributi definiti in **attr**, all'interno del processo corrente. Nell'ambito del corso utilizzeremo gli attributi di default2 specificando **NULL come valore per l'argomento attr**. In caso di successo, l'**ID del thread creato** viene memorizzato nella porzione di memoria puntata da **thread**. Il thread viene creato eseguendo la **funzione start_routine**, che prenderà come unico argomento il buffer puntato da **arg**. Essa potrà opzionalmente restituire un puntatore tramite `pthread_exit()` o `return`; nel secondo caso, l'effetto è quello di una chiamata implicita a `pthread_exit()` con il valore della `return` come argomento. Maschera e gestori dei segnali saranno ereditati dal thread creante.
- **Valore di ritorno:**
 - In caso di **successo**, la funzione restituisce **0**, e il nuovo thread eredita maschera e gestore dei segnali dal thread creante. In caso di **fallimento**, verrà restituito un codice di errore che ne indicherà la causa (la variabile `errno` non verrà aggiornata).

2.4 Uscita da un thread

void pthread_exit(void *value_ptr);

- **Argomenti e funzionamento**
 - La funzione termina il thread in cui viene invocata, e rende il valore del puntatore **value_ptr** disponibile ad ogni operazione di join che verrà fatta sul thread da terminare.
 - Al momento della terminazione non vengono rilasciate automaticamente risorse che sono visibili nel resto del processo, quali semafori e descrittori di file.
 - Per tutti i thread diversi dal main thread, un'istruzione return equivale ad una chiamata implicita a pthread_exit(), usando il valore restituito come argomento della chiamata.
 - Per il main thread, un'istruzione return equivale invece ad una chiamata implicita ad exit(), usando il valore restituito come exit status. Una chiamata a pthread_exit() nel main thread permette invece al processo di proseguire nell'esecuzione finché tutti gli altri thread attivi termineranno spontaneamente.
 - Una volta che il thread chiamante è terminato, accedere ad indirizzi di variabili locali a quel thread dà luogo ad un comportamento indefinito. Di conseguenza, riferimenti a variabili locali non vanno contemplati come possibile valore per value_ptr. In assenza di un valore da restituire, si può utilizzare NULL come argomento.
- **Valore di ritorno:**
 - Non potendo ritornare nel chiamante, la funzione pthread_exit() non restituisce alcun valore, né sono previsti codici di errore.

2.5 Sincronizzazione tra thread

Di default i thread sono joinable, ossia è possibile attenderne la terminazione in modo esplicito da altri thread e quindi leggerne il valore restituito (se esso è previsto). Per questo motivo l'implementazione sottostante mantiene una serie di strutture dati associate al thread, che restano disponibili anche dopo la terminazione : quando non sono previste operazioni di join su un thread, è buona pratica liberare esplicitamente queste risorse.

int pthread_join(pthread_t thread, void **value_ptr);

- **Argomenti e funzionamento**
 - La funzione sospende l'esecuzione del thread chiamante fino alla terminazione del thread, a meno che quest'ultimo non sia già terminato. Quando la chiamata ha successo ed il valore specificato **per l'argomento value_ptr** è diverso da NULL, **il valore passato a pthread_exit() nel thread terminante viene scritto nella locazione di memoria puntata da value_ptr.**
- **Valore di ritorno:**
 - In caso di successo, **la funzione restituisce 0.** Altrimenti, **viene restituito un codice di errore che indica la causa del fallimento.**

int pthread_detach(pthread_t thread);

- **Argomenti e funzionamento**
 - La funzione viene usata per indicare all'implementazione sottostante che lo storage previsto per il **thread thread** potrà essere reclamato quando quel thread terminerà.
- **Valore di ritorno:**
 - In caso di **successo, la funzione restituisce 0.** Altrimenti, **viene restituito un codice di errore che indica la causa del fallimento.**

Semafori

FUNZIONI

Apertura e chiusura di un semaforo anonimo

int sem_init(sem_t *sem, int pshared, unsigned int value);

- **Argomenti e funzionamento**
 - Inizializza il semaforo anonimo puntato da **sem** con valore iniziale **value**. Nell'ambito del corso abbiamo assunto che il valore scelto per **pshared** sia **0**. Se il semaforo dovesse essere condiviso tra più processi senza una relazione padre-figlio deve essere posto a 1.
- **Ritorno**
 - In caso di **successo**, inizializza il semaforo puntato da **sem** come richiesto e **restituisce 0**.
 - Altrimenti, viene restituito **-1** e la **variabile errno** indicherà la causa dell'errore.

int sem_destroy(sem_t *sem);

- **Argomenti:**
 - Distrugge il semaforo anonimo puntato da **sem**. Se invocato su un semaforo named, il suo comportamento non è definito.
- **Valore di ritorno:**
 - In caso di **successo**, **viene restituito 0**. Altrimenti, viene restituito **-1** e la **variabile errno** indicherà la causa dell'errore.

3.2 Operazioni sui semafori

int sem_wait(sem_t *sem);

- **Argomenti e Funzionamento:**
 - La funzione decrementa di un'unità il contatore associato al semaforo puntato da **sem**.
 - Se il decremento producesse un valore negativo per il contatore, la chiamata resta bloccata in attesa che un altro thread incrementi il contatore, a meno che essa non venga interrotta dall'arrivo di un segnale.
- **Valore di ritorno:**
 - In caso di **successo**, **viene restituito 0**. Altrimenti, **viene restituito -1** e la **variabile errno** indicherà la causa dell'errore (e.g., EINTR).

int sem_post(sem_t *sem);

- **Argomenti e Funzionamento**
 - La funzione incrementa di un'unità il contatore associato al semaforo puntato da **sem**.
 - Se nel processo vi sono thread bloccati in attesa che il semaforo puntato da **sem** venga incrementato, uno di essi verrà sbloccato e la sua **sem_wait()** ritornerà con successo.
- **Valore di ritorno:**
 - In caso di **successo**, **la sem_post() restituisce 0**. Altrimenti, viene **restituito -1** e la **variabile errno** indicherà la causa dell'errore.

int sem_getvalue(sem_t *sem, int *sval);

- **Argomenti e Funzionamento**
 - Memorizza nell'indirizzo **sval** il valore del contatore associato al semaforo puntato da **sem**, senza alterare lo stato del semaforo.
 - Se la coda di thread in attesa non è vuota, nei sistemi Linux il valore scritto in **sval** sarà zero (su altri sistemi il valore potrà invece corrispondere al numero di thread in coda).
- **Valore di ritorno:**
 - In caso di **successo**, **viene restituito 0**. Altrimenti, **viene restituito -1** e la **variabile errno** indicherà la causa dell'errore.

3.3 Apertura di un semaforo named

Per i semafori named si **includa** `<fcntl.h>` per le costanti da usare per il parametro `oflag` (e.g., `O_CREAT`, `O_EXCL`), ed eventualmente `<sys/stat.h>` per usare le costanti previste per argomenti `mode_t` in luogo di una maschera dei permessi in codifica ottale.

sem_t* sem_open(const char* name, int oflag);
sem_t* sem_open(const char* name, int oflag,
mode_t mode, unsigned int value);

- **Argomenti e Funzionamento**

- La funzione crea un semaforo named o ne apre uno esistente.
- **Il semaforo è identificato univocamente da name, che per convenzione è una stringa del tipo /nomeSemaforo**; ossia, una stringa null-terminated composta da massimo 251 caratteri, di cui un simbolo backslash / iniziale, seguito da uno o più caratteri, nessuno dei quali può essere un backslash .5
- **Il parametro oflag** specifica i flag (definiti in `<fcntl.h>`) che controllano il modo in cui deve operare la chiamata. In assenza di flag da specificare, il valore da utilizzare è 0.
 - Se il flag **O_CREAT** è specificato, il semaforo viene creato se non è già presente nel sistema. Se **O_CREAT** ed **O_EXCL** sono specificati contemporaneamente (tramite OR bit a bit: **O_CREAT | O_EXCL**), viene restituito un errore qualora il semaforo per il name specificato sia già esistente.
 - **Quando O_CREAT è specificato**, sono necessari due ulteriori argomenti per la funzione.
- **Il parametro mode** specifica i permessi di creazione per il semaforo, analogamente a quanto avviene per i file. Per operare correttamente, un processo ha bisogno di poter accedere al semaforo sia in lettura che in scrittura. Per la specifica dei permessi si può utilizzare la 5 codifica ottale (e.g., **0660**) oppure una delle macro definite in `<sys/stat.h>`.
- **Il parametro value** specifica il valore iniziale per il semaforo da creare. Si tenga a mente che qualora il semaforo named sia già presente nel sistema, i parametri mode e value saranno ignorati.

- **Valore di ritorno:**

- In caso di **successo**, **sem_open()** restituisce l'indirizzo del nuovo semaforo, che potrà essere usato come argomento per operazioni quali `sem_wait()`, `sem_getvalue()` e `sem_post()`. L'implementazione alloca la struttura `sem_t` in una memoria condivisa gestita dal kernel, per cui il puntatore continua ad essere valido in caso di `fork()`.
- In caso di errore, **sem_open()** restituisce **SEM_FAILED**, e la variabile `errno` indicherà la causa dell'errore (e.g., `EEXIST`).

3.4 Chiusura e distruzione di un semaforo named

int sem_close(sem_t *sem);

- **Argomenti e Funzionamento**

- Chiude il semaforo named puntato da **sem**, permettendo che le risorse allocate dal sistema operativo per il processo che ha aperto il semaforo vengano liberate.

- **Valore di ritorno:**

- In caso di **successo**, viene restituito **0**. Altrimenti, viene restituito **-1** e la variabile `errno` indicherà la causa dell'errore.

int sem_unlink(const char *name);

- **Argomenti e Funzionamento**
 - Rimuove il semaforo named identificato univocamente da name.
 - Il nome associato al semaforo viene immediatamente reso libero nel sistema, mentre il semaforo viene distrutto non appena gli altri processi che hanno aperto il semaforo l'avranno chiuso.
 - La funzione è MT-Safe: può essere eseguita in modo safe in programmi multi-thread .6
- **Valore di ritorno**
 - In caso di **successo, viene restituito 0**. Altrimenti, **viene restituito -1** e la variabile **errno** indicherà la causa dell'errore.

ESEMPIO DI APPLICAZIONE SEMAFORI NORMALI

Per le named è la stessa cosa ma si dovrebbe fare una **sem_open** speciale

```
//Nel caso 1-1 facciamo un solo semaforo per l'accesso. l'ordine di semafori
//è stato sem_wait(sem_access) - sem_post(sem_produced) - sem_wait(sem_produced) -
sem_post(sem_access)
//come se fosse un unico flusso. questo non sarà possibile successivamente

//Nel caso 1-M. faccio due sem_access, sem_access1 e sem_access2, per fare in modo che
//non ci sia busy waiting tra i consumatori e i produttori
//ordine sem_wait(sem_access1) - sem_post(sem_produced) - sem_wait(sem_produced) -
sem_wait(sem_access2) -
//- sem_post(sem_access2) - sem_post(sem_access1). Nota come access1 sta all'inizio ed
alla fine

//Caso N-1: Semafori. ordine: sem_wait(sem_limit) - sem_wait(sem_access1) -
sem_post(sem_access1) - sem_post(sem_produced) - sem_wait(sem_produced) -
sem_post(sem_limit)
//sem_limit è un semaforo che mi dice quanti posti liberi ci sono nel buffer

//CASO GLOBALE!! N consumer e M producer

#define BUFFER_SIZE          128
#define INITIAL_DEPOSIT      0
#define MAX_TRANSACTION      1000
#define NUM_CONSUMERS        2
#define NUM_PRODUCERS        4
#define PRNG_SEED            0

#define NUM_OPERATIONS        400
#define OPS_PER_CONSUMER      (NUM_OPERATIONS/NUM_CONSUMERS)
#define OPS_PER_PRODUCER      (NUM_OPERATIONS/NUM_PRODUCERS)

// struct used to specify arguments for a thread
typedef struct {
    int threadId;
    int numOps;
} thread_args_t;

// shared data
```

```

int transactions[BUFFER_SIZE];
int deposit = INITIAL_DEPOSIT;
int read_index, write_index;

//Semafori
sem_t sem_access1, sem_access2, sem_produced, sem_limit;

// producer thread
void* performTransactions(void* x) {
    thread_args_t* args = (thread_args_t*)x;

    printf("Starting producer thread %d\n", args->threadId);

    while (args->numOps > 0) {
        if (sem_wait(&sem_limit) == -1){
            fprintf(stderr, "Errore nella sem_wait del semaforo sem_limit nel
performTransaction... error %s\n", errno);
            exit(EXIT_FAILURE);
        }
        if (sem_wait(&sem_access1) == -1){
            fprintf(stderr, "Errore nella sem_wait del semaforo sem_access nel
performTransaction... error %s\n", errno);
            exit(EXIT_FAILURE);
        }
        // produce the item
        int currentTransaction = performRandomTransaction();

        // write the item and update write_index accordingly
        transactions[write_index] = currentTransaction;
        write_index = (write_index + 1) % BUFFER_SIZE;

        args->numOps--;
        //printf("P %d\n", args->numOps);

        if (sem_post(&sem_access1) == -1){
            fprintf(stderr, "Errore nella sem_post del semaforo sem_access nel
performTransaction... error %s\n", errno);
            exit(EXIT_FAILURE);
        }
        if (sem_post(&sem_produced) == -1){
            fprintf(stderr, "Errore nella sem_post del semaforo sem_produced nel
performTransaction... error %s\n", errno);
            exit(EXIT_FAILURE);
        }
    }
    free(args);
    pthread_exit(NULL);
}

void* processTransactions(void* x) {
    thread_args_t* args = (thread_args_t*)x;

    printf("Starting consumer thread %d\n", args->threadId);

```

```

while (args->numOps > 0) {
    if (sem_wait(&sem_produced) == -1){
        fprintf(stderr, "Errore nella sem_wait del semaforo sem_produced nel
processTransaction... error %s\n", errno);
        exit(EXIT_FAILURE);
    }
    if (sem_wait(&sem_access2) == -1){
        fprintf(stderr, "Errore nella sem_wait del semaforo sem_access nel
processTransaction... error %s\n", errno);
        exit(EXIT_FAILURE);
    }
    // consume the item and update (shared) variable deposit
    deposit += transactions[read_index];
    read_index = (read_index + 1) % BUFFER_SIZE;
    if (read_index % 100 == 0)
        printf("After the last 100 transactions balance is now %d.\n", deposit);

    args->numOps--;
    //printf("C %d\n", args->numOps);

    if (sem_post(&sem_access2) == -1){
        fprintf(stderr, "Errore nella sem_post del semaforo sem_access nel
processTransaction... error %s\n", errno);
        exit(EXIT_FAILURE);
    }
    if (sem_post(&sem_limit) == -1){
        fprintf(stderr, "Errore nella sem_post del semaforo sem_limit nel
processTransaction... error %s\n", errno);
        exit(EXIT_FAILURE);
    }
}

free(args);
pthread_exit(NULL);
}

int main(int argc, char* argv[]) {
    // initialize read and write indexes
    read_index = 0;
    write_index = 0;

    int ret;
    pthread_t producer[NUM_PRODUCERS], consumer[NUM_CONSUMERS];

    //inizializzo semafori
    if (sem_init(&sem_access1, 0, 1) == -1){
        fprintf(stderr, "Errore nell'inizializzazione del semaforo sem_access... error
%s\n", errno);
        exit(EXIT_FAILURE);
    }
    if (sem_init(&sem_access2, 0, 1) == -1){
        fprintf(stderr, "Errore nell'inizializzazione del semaforo sem_access... error
%s\n", errno);

```

```

        exit(EXIT_FAILURE);
    }
    if (sem_init(&sem_produced, 0, 0) == -1){
        fprintf(stderr, "Errore nell'inizializzazione del semaforo sem_produced... error
%s\n", errno);
        exit(EXIT_FAILURE);
    }
    if (sem_init(&sem_limit, 0, BUFFER_SIZE) == -1){
        fprintf(stderr, "Errore nell'inizializzazione del semaforo sem_limit... error
%s\n", errno);
        exit(EXIT_FAILURE);
    }

    int i;
    //invia i producers
    for (i=0; i<NUM_PRODUCERS; ++i) {
        thread_args_t* arg = malloc(sizeof(thread_args_t));
        arg->threadId = i;
        arg->numOps = OPS_PER_PRODUCER;

        ret = pthread_create(&producer[i], NULL, performTransactions, arg);
        if (ret != 0) { fprintf(stderr, "Error %d in pthread_create\n", ret);
exit(EXIT_FAILURE); }
    }

    int j;
    //invia i consumers
    for (j=0; j<NUM_CONSUMERS; ++j) {
        thread_args_t* arg = malloc(sizeof(thread_args_t));
        arg->threadId = j;
        arg->numOps = OPS_PER_CONSUMER;

        ret = pthread_create(&consumer[j], NULL, processTransactions, arg);
        if (ret != 0) { fprintf(stderr, "Error %d in pthread_create\n", ret);
exit(EXIT_FAILURE); }
    }

    // join on threads
    //recupera i producers
    for (i=0; i<NUM_PRODUCERS; ++i) {
        ret = pthread_join(producer[i], NULL);
        if (ret != 0) { fprintf(stderr, "Error %d in pthread_join\n", ret);
exit(EXIT_FAILURE); }
    }
    //recupera i consumers
    for (j=0; j<NUM_CONSUMERS; ++j) {
        ret = pthread_join(consumer[j], NULL);
        if (ret != 0) { fprintf(stderr, "Error %d in pthread_join\n", ret);
exit(EXIT_FAILURE); }
    }

    printf("Final value for deposit: %d\n", deposit);

    //distruggo i semafori

```

```
    if (sem_destroy(&sem_access1) == -1){
        fprintf(stderr, "Errore nella distruzione del semaforo sem_access... error %s\n",
errno);
        exit(EXIT_FAILURE);
    }
    if (sem_destroy(&sem_access2) == -1){
        fprintf(stderr, "Errore nella distruzione del semaforo sem_access... error %s\n",
errno);
        exit(EXIT_FAILURE);
    }
    if (sem_destroy(&sem_produced) == -1){
        fprintf(stderr, "Errore nella distruzione del semaforo sem_produced... error
%s\n", errno);
        exit(EXIT_FAILURE);
    }
    if (sem_destroy(&sem_limit) == -1){
        fprintf(stderr, "Errore nella distruzione del semaforo sem_limit... error %s\n",
errno);
        exit(EXIT_FAILURE);
    }
    exit(EXIT_SUCCESS);
}
```

Shared Memory

FUNZIONI

int shm_open(const char pathname, int flags, mode_t mode);*

int shm_open(const char pathname, int flags);*

- **argomenti e funzionamento**
 - La funzione restituisce un descrittore per la memoria condivisa.
 - Il secondo argomento **flags** deve obbligatoriamente contenere una delle seguenti modalità di accesso: **O_RDONLY** (sola lettura), **O_WRONLY** (sola scrittura), o **O_RDWR** (lettura e scrittura). I flag descritti sono definiti in `<fcntl.h>`.
 - Se **O_CREAT** è specificato, la shared memory viene creata se non è già presente nel sistema; se **O_CREAT** ed **O_EXCL** sono entrambi specificati, viene restituito un errore se la shared memory che si desidera creare è già esistente nel sistema.
 - Quando **O_CREAT** viene specificato, è necessario fornire alla funzione tramite l'argomento **mode** i permessi di creazione per la memoria (se non già presente nel sistema). Per la specifica dei permessi si può utilizzare la **codifica ottale** (e.g., **0660**) oppure una delle macro definite in `<sys/stat.h>`.
- **Valore di ritorno**
 - In caso di **successo**, viene restituito il descrittore così aperto (un intero ≥ 0).
 - Altrimenti, viene restituito **-1** e la variabile **errno** indicherà la causa dell'errore (e.g., **EINTR**).

int ftruncate(int fd, off_t length);

- **Argomenti e funzionamento**
 - La funzione dimensiona la memoria condivisa a cui fa riferimento **fd** a una dimensione di **length** bytes.
 - Se la memoria condivisa in precedenza fosse più grande di **length**, i dati extra andrebbero persi.
 - Se la memoria in precedenza era più corta, viene estesa e la parte estesa viene letta come byte null (`'\0'`).
- **Valore di ritorno**
 - In caso di **successo**, viene restituito il valore **0**.
 - Altrimenti, viene restituito **-1** e la variabile **errno** indicherà la causa dell'errore (e.g., **EINTR**).

void mmap(void * addr, size_t length, int prot, int flags, int fd, off_t offset);*

- **Argomenti e funzionamento**
 - La funzione mappa la shared memory nella memoria riservata al processo.
 - Il parametro **addr** permette di suggerire al kernel dove posizionare la memoria condivisa. Se **NULL** o **0** (per noi sempre **0**) il kernel decide autonomamente. Il parametro **length** rappresenta la dimensione della memoria condivisa.
 - Il parametro **prot** descrive la protezione desiderata per il mapping della memoria (non deve essere in conflitto con i parametri di apertura della memoria).
 - Può essere **PROT_NONE** o il bitwise OR di uno dei seguenti flag:
 - **PROT_EXEC** le pagine possono essere eseguite
 - **PROT_READ** le pagine possono essere lette
 - **PROT_WRITE** le pagine possono essere scritte

- **PROT_NONE** non si può accedere alle pagine.
- Il parametro **flags** determina se le modifiche alla memoria sono visibili agli altri processi con cui la memoria è condivisa. **Per il nostro corso useremo sempre il valore MAP_SHARED**
- Il parametro **fd** è il descrittore della memoria condivisa.
- Il parametro **offset** permette di mappare la memoria condivisa da una posizione diversa da quella iniziale. **offset** deve essere un multiplo della page size.
- **Valore di ritorno**
 - Per noi **0** In caso di successo, viene restituito il puntatore all'area di memoria dove risiede la shared memory.
 - Altrimenti, viene restituito **MAP_FAILED** e la variabile **errno** indicherà la causa dell'errore (e.g., **EINTR**)

void* munmap(void * addr, size_t length);

- **Argomenti e funzionamento**
 - La funzione cancella il mapping tra il processo e la memoria condivisa.
 - Successivi tentativi di accesso tramite il puntatore falliranno.
- **Valore di ritorno**
 - In caso di **successo, viene restituito il valore 0.**
 - Altrimenti, **viene restituito -1** e la variabile **errno** indicherà la causa dell'errore (e.g., **EINTR**).

void* shm_unlink(const char* pathname);

- **Argomenti e funzionamento**
 - La funzione rimuove una memoria condivisa.
 - Una volta che tutti i processi hanno fatto l'unmap della memoria, disalloca e distrugge il contenuto della regione di memoria associata.
 - Successivi tentativi di aprire la memoria falliranno (a meno che non venga usata l'opzione **O_CREAT**)
- **Valore di ritorno**
 - In caso di **successo, viene restituito il valore 0.**
 - Altrimenti, **viene restituito -1** e la variabile **errno** indicherà la causa dell'errore (e.g., **EINTR**)

ESEMPIO DI UTILIZZO

PRODUCER CONSUMER CON SEMAFORI

producer.c

```
struct shared_memory {
    int buf [BUFFER_SIZE];
    int read_index;
    int write_index;
};

struct shared_memory *myshm_ptr;
int fd_shm;

sem_t *sem_empty, *sem_filled, *sem_cs;

void initMemory() {
    shm_unlink(SH_MEM_NAME);

    if((fd_shm = shm_open(SH_MEM_NAME, O_CREAT | O_EXCL, 0666)) < 0){
        handle_error("producer: initMemory shm_open error");
    }

    if(ftruncate (fd_shm, sizeof(struct shared_memory))){
        handle_error("producer: initMemory ftruncate error");
    }

    if((myshm_ptr = mmap (NULL, sizeof(struct shared_memory), PROT_READ | PROT_WRITE,
MAP_SHARED, fd_shm, 0)) == MAP_FAILED){
        handle_error("consumer: initMemory mmap error");
    }
    memset(myshm_ptr, 0, sizeof(struct shared_memory));
}

void closeMemory() {
    if(munmap(myshm_ptr, sizeof(struct shared_memory))){
        handle_error("producer: closeMemory munmap error");
    }
    if(close(fd_shm)){
        handle_error("producer: closeMemory close (fd) error");
    }
    if(shm_unlink(SH_MEM_NAME)){
        handle_error("producer: closeMemory shm_unlink error");
    }
}
```



```

    }
}

void initSemaphores() {}

void closeSemaphores() {}

static inline int performRandomTransaction() {}

void produce(int id, int numOps) {
    int localSum = 0;
    while (numOps > 0) {
        // producer, just do your thing!
        int value = performRandomTransaction();

        int ret = sem_wait(sem_empty);
        if (ret) handle_error("sem_wait empty\n");

        ret = sem_wait(sem_cs);
        if (ret) handle_error("sem_wait cs");

        myshm_ptr->buf[myshm_ptr->write_index] = value;
        myshm_ptr->write_index = (myshm_ptr->write_index + 1) % BUFFER_SIZE;

        ret = sem_post(sem_cs);
        if (ret) handle_error("sem_post cs");

        ret = sem_post(sem_filled);
        if (ret) handle_error("sem_post filled");

        localSum += value;
        numOps--;
    }
    printf("Producer %d ended. Local sum is %d\n", id, localSum);
}

int main(int argc, char** argv) {
    srand(PRNG_SEED);
    initSemaphores();
    initMemory(); //definita sopra

    int i, ret;
    for (i=0; i<NUM_PRODUCERS; ++i) {
        pid_t pid = fork();
        if (pid == -1) {

```

```

        handle_error("fork");
    } else if (pid == 0) {
        produce(i, OPS_PER_PRODUCER);
        _exit(EXIT_SUCCESS);
    }
}

for (i=0; i<NUM_PRODUCERS; ++i) {
    int status;
    ret = wait(&status);
    if (ret == -1) handle_error("wait");
    if (WEXITSTATUS(status)) handle_error_en(WEXITSTATUS(status), "child crashed");
}

printf("Producers have terminated. Exiting...\n");
closeSemaphores();
closeMemory(); //definita sopra
exit(EXIT_SUCCESS);
}

```

consumer.c

```
struct shared_memory {
    int buf [BUFFER_SIZE];
    int read_index;
    int write_index;
};

struct shared_memory *myshm_ptr;
int fd_shm;

sem_t *sem_empty, *sem_filled, *sem_cs;

void openMemory() {
    shm_unlink(SH_MEM_NAME);
    if((fd_shm = shm_open(SH_MEM_NAME, O_CREAT | O_EXCL , 0666)) < 0){
        handle_error("consumer: openMemory shm_open error");
    }
    if((myshm_ptr = mmap(NULL, sizeof(struct shared_memory), PROT_READ | PROT_WRITE,
MAP_SHARED, fd_shm, 0)) == MAP_FAILED){
        handle_error("consumer: openMemory mmap error");
    }
    //A differenza del producer, non fa la ftruncate ed il memset
}

void closeMemory() {
    if(munmap(myshm_ptr, sizeof(struct shared_memory))){
        handle_error("consumer: closeMemory munmap error");
    }
    if(close(fd_shm)){
        handle_error("consumer: closeMemory close (fd) error");
    }
    //non fa la shm_unlink per rendere accessibile la memoria ad altri consumer
}

void openSemaphores() {}

void closeAndDestroySemaphores() {}

void consume (int id, int numOps) {
    int localSum = 0;
    while (numOps > 0) {
        int ret = sem_wait(sem_filled);
```

```

    if (ret) handle_error("sem_wait filled");

    ret = sem_wait(sem_cs);
    if (ret) handle_error("sem_wait cs");

    int value = myshm_ptr->buf[mysshm_ptr->read_index];
    myshm_ptr->read_index = (mysshm_ptr->read_index + 1) % BUFFER_SIZE;

    ret = sem_post(sem_cs);
    if (ret) handle_error("sem_post cs");

    ret = sem_post(sem_empty);
    if (ret) handle_error("sem_post empty");

    localSum += value;
    numOps--;
}
printf("Consumer %d ended. Local sum is %d\n", id, localSum);
}

int main (int argc, char** argv) {
    openSemaphores();
    openMemory(); //definita sopra
    int i;
    for (i=0; i<NUM_CONSUMERS; ++i) {
        pid_t pid = fork();
        if (pid == -1) {
            handle_error("fork");
        } else if (pid == 0) {
            consume(i, OPS_PER_CONSUMER);
            _exit(EXIT_SUCCESS);
        }
    }
    for (i=0; i<NUM_CONSUMERS; ++i) {
        int status;
        wait(&status);
        if (WEXITSTATUS(status)) handle_error("child crashed");
    }
    printf("Consumers have terminated. Exiting...\n");
    closeAndDestroySemaphores();
    closeMemory(); //definita sopra
    exit(EXIT_SUCCESS);
}

```

PIPE e FIFO

FUNZIONI

//pipe normali, più avanti ci sono le NAMED PIPE dette anche FIFO

int pipe(int pipefd[2]);

- **Argomenti e funzionamento**
 - Crea una pipe, un canale di comunicazione unidirezionale che può essere usato per comunicazioni inter-processo.
 - **L'array pipefd** è usato per restituire i due descrittori relativi alla pipe:
 - **pipefd[0]** per la lettura
 - **pipefd[1]** per la scrittura.
 - Per usarla, bisogna includere. <unistd.h>.
- **Valori di ritorno**
 - In caso di **successo**, la funzione ritorna **0**.
 - In caso di **fallimento**, viene restituito **-1** ed errno viene impostata con un codice che caratterizza l'errore avvenuto.

int dup(int oldfd);

- **Argomenti e funzionamento**
 - Crea una copia del descrittore **oldfd**.
 - Per il nuovo descrittore, viene scelto il primo valore di descrittore non utilizzato (quello con valore minimo nella tabella dei descrittori del processo). Per usarla, bisogna includere <unistd.h>
- **Valori di ritorno**
 - In caso di **successo**, la funzione ritorna il nuovo descrittore.
 - In caso di **fallimento**, viene restituito **-1** ed errno viene impostata con un codice che caratterizza l'errore avvenuto.

int dup2(int oldfd, int newfd);

- **Argomenti e funzionamento**
 - Crea una copia del descrittore **oldfd**.
 - Per il nuovo descrittore, viene scelto il valore **newfd** passato come secondo parametro.
 - Se necessario, **newfd** viene prima chiuso.
 - Per usarla, bisogna includere <unistd.h>.
- **Valori di ritorno**
 - In caso di **successo**, la funzione ritorna il nuovo descrittore.
 - In caso di **fallimento**, viene restituito **-1** ed errno viene impostata con un codice che caratterizza l'errore avvenuto.

//FIFO NAMED

int mkfifo(const char *pathname, mode_t mode);

- **Argomenti e funzionamento**
 - Crea una **FIFO con nome pathname**. Il parametro mode specifica i permessi della FIFO stessa (e.g., 0660).
- **Valori di ritorno**
 - Per usarla, bisogna includere <sys/types.h> e <sys/stat.h>.
 - In caso di **successo**, la funzione ritorna **0**.
 - In caso di **fallimento**, viene restituito **-1** ed errno viene impostata con un codice che caratterizza l'errore avvenuto (e.g., EEXIST)

ESEMPIO DI UTILIZZO PIPE, LE FIFO SI USANO CON I PROCESSI

```
int pipefd[2];

int write_to_pipe(int fd, const void *data, size_t data_len) {
    //SCRITTURA IDENTICA COME SU FILE
    //USANDO LE PIPE

    int written_bytes = 0;
    int bytes_left = data_len;
    while (bytes_left > 0) {
        int ret = write(fd, data+written_bytes, bytes_left);
        if (ret == -1) {
            if (errno == EINTR) continue;
            else exit(EXIT_FAILURE);
        }
        bytes_left -= ret;
        written_bytes += ret;
    }
    //if (written_bytes == data_len) printf("sono stati scritti tutti i %d bytes",
written_bytes);
    return written_bytes;
}

int read_from_pipe(int fd, void *data, size_t data_len) { //LETTURA IDENTICA COME SU FILE
    //USANDO LE PIPE

    int read_bytes = 0;
    int bytes_left = data_len;
    while (bytes_left > 0) {
        if (bytes_left > 0) {
            int ret = read(fd, data+read_bytes, bytes_left);
            if (ret == -1) {
                if (errno == EINTR) continue;
                exit(EXIT_FAILURE);
            }
            if (ret == 0) break;

            bytes_left -= ret;
            read_bytes += ret;
        }
    }
    return read_bytes;
}

void reader(int reader_id, sem_t* read_mutex) {
    int data[MSG_ELEMS];
    int i, ret;
    printf("[READER_%d] processo reader creato.\n", reader_id);

    if (close(pipefd[1])) handle_error("errore chiusura");

    for (i = 0; i < MSG_COUNT/READERS_COUNT; i++) {
```

```

        ret = sem_wait(read_mutex);
        if(ret) handle_error("error waiting on read mutex");

FUNZIONE SOPRA-→ read_from_pipe(pipefd[0], data, sizeof(data)); // sizeof(data) ==
MSG_ELEMS * sizeof(int)

        ret = sem_post(read_mutex);
        if(ret) handle_error("error posting on read mutex");

        printf("[CHILD_%d] Letto msg #%d con valore %d\n", reader_id, i, data[0]);
        if (!is_msg_ok(data, MSG_ELEMS))
            printf("corrupted message!!!\n");
    }

    ret = sem_close(read_mutex);
    if(ret) handle_error("error closing read mutex");

    ret = close(pipefd[0]);
    if(ret) handle_error("error closing pipe");
}

void writer(int writer_id, sem_t* write_mutex) {
    int data[MSG_ELEMS];
    int i,ret;
    printf("[WRITER_%d] processo writer creato.\n", writer_id);

    if (close(pipefd[0])) handle_error("errore chiusura file descriptor lettura");

    for (i = 0 ; i < MSG_COUNT/WRITERS_COUNT; i++) {
        create_msg(data, MSG_ELEMS, i);
        ret = sem_wait(write_mutex);
        if(ret) handle_error("error waiting on write mutex");

        write_to_pipe(pipefd[1], data, sizeof(data)); // sizeof(data) == MSG_ELEMS *
sizeof(int)

        ret = sem_post(write_mutex);
        if(ret) handle_error("error posting on write mutex");

        printf("[WRITER_%d] Inviato il msg #%d\n", writer_id, i);
    }

    ret = sem_close(write_mutex);
    if(ret) handle_error("error closing write mutex");

    if (close(pipefd[1])) handle_error("errore chiusura file descriptor scrittura");
}

int main(int argc, char* argv[]) {
    int ret,i;
    pid_t pid;

```

```

sem_unlink(READ_MUTEX);
sem_t *read_mutex = sem_open(READ_MUTEX, O_CREAT|O_EXCL, 0600, 1);
if(read_mutex == SEM_FAILED) handle_error("Error Creating Read Mutex");

sem_unlink(WRITE_MUTEX);
sem_t *write_mutex = sem_open(WRITE_MUTEX, O_CREAT|O_EXCL, 0600, 1);
if(write_mutex == SEM_FAILED) handle_error("Error Creating Write Mutex");

if (pipe(pipefd)==-1) handle_error("errore apertura pipe!");

for (i = 0; i < READERS_COUNT; i++) {
    pid = fork();
    if (pid == -1) handle_error("Error creating reader");
    if (pid == 0) {
        reader(i,read_mutex);
        _exit(0);
    }
}

ret = sem_close(read_mutex);
if(ret) handle_error("Error closing read mutex");

for (i = 0; i < WRITERS_COUNT; i++) {
    pid = fork();
    if (pid == -1) handle_error("error creating reader");
    if (pid == 0) {
        writer(i,write_mutex);
        _exit(0);
    }
}

ret = sem_close(write_mutex);
if(ret) handle_error("Error closing write mutex");

// shutdown phase
for (i = 0 ; i < READERS_COUNT + WRITERS_COUNT; i++) {
    int status;
    ret = wait(&status);
    if(ret == 1) handle_error("error waiting for a child to terminate");
    if (WEXITSTATUS(status)) handle_error("child process died unexpectedly");
}

printf("[PARENT] processi figlio terminati.\n");

ret = sem_unlink(READ_MUTEX);
if(ret) handle_error("error removing read mutex");

ret = sem_unlink(WRITE_MUTEX);
if(ret) handle_error("error removing write mutex");

return 0;
}

```


Socket e Network

FUNZIONI

int socket(int family, int type, int protocol);

- Crea una socket, ossia un endpoint di comunicazione
- **Argomenti**
 - **family**: per i nostri scopi, **AF_INET** (vedi struttura dati struct sockaddr_in)
 - **type**: per i nostri scopi, **SOCK_STREAM** (protocollo **TCP**) Ne esistono altre, es: **SOCK_DGRAM** (protocollo **UDP**)
 - **protocol**: per i nostri scopi, **0**
- **Valore di ritorno**
 - In caso di **successo**, il descrittore della socket
 - In caso di **errore**, -1, errno è settato

int bind(int fd, const struct sockaddr *addr, socklen_t len);

- Assegna un indirizzo ad una socket
- **Argomenti**
 - **fd**: descrittore della socket (restituito da socket())
 - **addr**: puntatore ad una struttura dati che specifica l'indirizzo
 - Per i nostri scopi: la struttura **struct sockaddr_in** va castata a **struct sockaddr**
 - **len**: dimensione della struttura dati puntata da addr
- **Valore di ritorno**
 - In caso di successo, 0
 - In caso di errore, -1 , errno è settato

int listen(int sockfd, int backlog);

- Marca la socket come passiva, i.e., specifica che può essere usata per accettare connessioni tramite la funzione **accept()**
- **Argomenti**
 - **sockfd**: descrittore della socket (restituito da socket())
 - **backlog**: lunghezza massima della coda per le connessioni
 - Se una connessione arriva quando la coda è piena, la connessione viene rifiutata
- **Valore di ritorno**
 - In caso di successo, 0
 - In caso di errore, -1 , errno è settato

int accept(int fd, struct sockaddr *addr, socklen_t *len);

- Accetta una connessione su una socket in ascolto
 - È una chiamata bloccante: rimane in attesa di connessioni
- **Argomenti**
 - **fd**: descrittore della socket (restituito da socket())
 - **addr**: puntatore ad una struttura dati struct sockaddr che verrà riempita con le info della socket del client
 - **len**: puntatore ad un intero che verrà settato con la dimensione della struttura dati addr
- **Valore di ritorno**
 - In caso di successo, un descrittore per comunicare col client
 - In caso di errore, -1 , errno è settato

int close(int fd);

- Nel caso fd sia un descrittore di socket, chiude la socket stessa
 - **read()** successive dall'altro endpoint restituiranno 0 !!!
- **Argomenti**
 - **fd**: descrittore della socket (ritornato da socket())
- **Valore di ritorno**
 - in caso di successo, 0
 - In caso di errore, -1 , errno è settato

int connect(int fd, const struct sockaddr *addr, socklen_t l);

- Tenta una connessione su una socket in ascolto
- **Argomenti**
 - **fd**: descrittore della socket (ritornato da socket())
 - **addr**: puntatore ad una struttura dati struct sockaddr che descrive la socket alla quale connettersi (quella del server)
 - **l**: dimensione della struttura dati puntata da addr
- **Valore di ritorno**
 - In caso di successo, 0
 - In caso di errore, -1 , errno è settato

//Invio e ricezione TCP

ssize_t recv(int sockfd, void *buf, size_t len, int flags);

Riceve un messaggio via socket.

- **Argomenti:**
 - Il parametro **sockfd** indica il descrittore di socket da usare per la ricezione.
 - Il parametro **buf** è un puntatore all'area di memoria dove copiare il messaggio ricevuto.
 - Il parametro **len** indica il numero massimo di byte da leggere.
 - Nell'ambito del corso, il parametro **flags** va impostato a **0**. Così facendo, la **recv()** diventa equivalente alla **read()**.
- Per usarla bisogna includere **<sys/socket.h>**.
- **Ritorno:**
 - In caso di successo, la funzione ritorna il numero di byte effettivamente ricevuti. Se la connessione viene chiusa dall'altro lato della comunicazione, la funzione ritorna 0. In caso di fallimento, viene restituito -1 ed errno viene impostata con un codice che caratterizza l'errore avvenuto.

ssize_t send(int sockfd, const void *buf, size_t len, int flags);

Trasmette un messaggio via socket.

- **Argomenti**
 - Il parametro **sockfd** indica il descrittore di socket da usare per l'invio.
 - Il parametro **buf** è un puntatore all'area di memoria contenente messaggio da inviare.
 - Il parametro **len** indica il numero massimo di byte da inviare.
 - Nell'ambito del corso, il parametro **flags** va impostato a **0**. Così facendo, la **send()** diventa equivalente alla **write()**;
- Per usarla bisogna includere **<sys/socket.h>**.
- **Ritorno**
 - In caso di successo, la funzione ritorna il numero di byte effettivamente inviati. In caso di fallimento, viene restituito -1 ed errno viene impostata con un codice che caratterizza l'errore avvenuto.

//invio e ricezione UDP

**ssize_t sendto(int fd, const void *buf, size_t n, int flags,
const struct sockaddr *dest_addr, socklen_t l);**

- **fd, buf, n, flags**: come in **send()**
- **dest_addr, l**: come in **connect()**
- **Ritorna** il numero di byte realmente scritti, o -1 in caso di errore
- Default: **semantica bloccante**
 - • Se buffer di invio nel kernel non contiene spazio sufficiente per il messaggio da inviare, rimane bloccata in attesa...
- La funzione sendto() è definita in **sys/socket.h**

**ssize_t recvfrom(int fd, void *buf, size_t n, int flags,
struct sockaddr *src_addr, socklen_t *addrlen);**

- **fd, buf, n, flags**: come in **recv()**
- Se **src_addr** è una variabile la funzione inserisce le informazione del mittente nella variabile e inserisce in **addrlen** la lunghezza della struttura
 - Utile per rispondere o distinguere tra più possibili mittenti
- Se **src_addr** è **NULL**, non salva il mittente, **addrlen** non viene modificato e può essere **NULL**
 - Utile quando non ci interessa sapere chi ha inviato il messaggio
- **Ritorna** il numero di byte realmente letti, o -1 in caso di errore
- **Ritorna** 0 in caso di connessione chiusa
- Default: **semantica bloccante**
 - Se l'altro endpoint non invia nulla, rimane bloccata in attesa
 - Nel client andrebbe modificata la socket con un timeout, ma tralasciamo questo aspetto
 - Trasferisce i dati disponibili fino a quel momento nel buffer del kernel, entro il limite di n bytes, piuttosto che restare in attesa di ricevere l'intera quantità specificata...

struct in_addr: rappresenta un indirizzo IP a 32 bit

struct sockaddr_in: descrizione di una socket; al suo interno le informazioni principali sono:

- Famiglia dell'indirizzo (sin_family)
 1. Per i nostri scopi, **AF_INET**: protocollo IPv4
 2. Ne esistono altre, es: **AF_UNIX**, **AF_BLUETOOTH**
- Indirizzo IP (sin_addr.s_addr), per i nostri scopi:
 1. Lato server, **INADDR_ANY**: in ascolto su tutte le interfacce
 2. Lato client, specifica l'indirizzo IP del server
- Numero porta (sin_port)
 1. Bisogna rispettare l'ordine di trasmissione dei byte per la rete
 2. **sin_port = htons(port)** per invertire l'ordine dei bytes

MECCANISMO

SERVER SOCKET

- Come mettersi in ascolto su una porta nota?
 1. Creazione socket - funzione **socket()**
 2. Binding della socket su un indirizzo locale - funzione **bind()**
 3. Infine, mettersi in ascolto - funzione **listen()**
- • Come accettare una connessione da client?
 1. o Attesa di una connessione - funzione **accept()**
 - i. ➤ Una volta accettata una connessione, si ha a disposizione un descrittore di socket da usare per scambiare messaggi (tramite **send()/recv()**)
 2. Una volta terminato lo scambio di messaggi, la connessione col client va chiusa - funzione **close()**

CLIENT SOCKET

- • Come connettersi ad un server in ascolto?
 - o Creazione socket - funzione **socket()**
 - o Connessione al server - funzione **connect()**
 - o Una volta terminato lo scambio di messaggi, la connessione col client va chiusa - funzione **close()**

struct in_addr: rappresenta un indirizzo IP a 32 bit

struct sockaddr_in: descrizione di una socket; al suo interno le informazioni principali sono:

- Famiglia dell'indirizzo (sin_family)
 1. Per i nostri scopi, **AF_INET**: protocollo IPv4
 2. Ne esistono altre, es: **AF_UNIX**, **AF_BLUETOOTH**
- Indirizzo IP (sin_addr.s_addr), per i nostri scopi:
 1. Lato server, **INADDR_ANY**: in ascolto su tutte le interfacce
 2. Lato client, specifica l'indirizzo IP del server
- Numero porta (sin_port)
 1. Bisogna rispettare l'ordine di trasmissione dei byte per la rete
 2. **sin_port = htons(port)** per invertire l'ordine dei bytes
- Implementazione un protocollo client-server basato su messaggi di testo
 - o Il server è in ascolto su una porta nota
 - o Il client si connette al server
 - o Inizia uno scambio di messaggi di testo secondo uno schema predefinito («protocollo»)
 - o Protocollo di base
 - Il client invia una richiesta al server
 - Il server riceve la richiesta, la elabora, produce una risposta
 - Il server invia la risposta al client

➤ Funzione **ntohs()** (network-to-host-ushort)

- **uint16_t ntohs(uint16_t netshort);**
- Converte un ushort da network byte order a host byte order

Il numero di porta di una socket TCP può variare tra 0 e 65535

- la definizione del tipo generico uint16_t può essere inclusa tramite <arpa/inet.h> o più in generale <stdint.h>
- su Linux IA32 e x86_64 equivale ad un unsigned short
- le porte nel range 0-1023 richiedono privilegi di root

Struct_addr:

- • Un indirizzo IPv4 è rappresentato con struct in_addr
- • Il campo sin_addr di struct sockaddr_in è infatti di tipo struct in_addr. Reminder: variabili di tipo struct sockaddr_in vengono usate nella bind() e nella accept() lato server e nella connect() lato client
- • struct in_addr contiene il campo s_addr di tipo in_addr_t che rappresenta l'indirizzo in network byte order
- • Entrambe le strutture sono definite in <netinet/in.h>

in_addr_t inet_addr(const char *cp)

- Converte un indirizzo IPv4 dalla forma dotted string (x.y.z.w) al network byte order
- Il valore di ritorno viene di solito assegnato al campo s_addr di struct in_addr

const char *inet_ntop(int af, const void *src, char *dst, socklen_t n);

- Converte l'indirizzo di rete src della address family af in una stringa di lunghezza n e la copia in dst
- Ritorna un puntatore a dst, oppure NULL in caso di errore
- Le macro AF_INET e INET_ADDRSTRLEN possono essere usate rispettivamente per il primo e l'ultimo argomento
- Quanto vale INET_ADDRSTRLEN?

ESEMPIO DI APPLICAZIONE

SERVER

```
int main(int argc, char* argv[]) {
    int ret;
    int socket_desc, client_desc;
    // some fields are required to be filled with 0
    struct sockaddr_in server_addr = {0}, client_addr = {0}; //STRUCT PER IL SERVER e CLIENT

    int sockaddr_len = sizeof(struct sockaddr_in); // we will reuse it for accept()

    //SOCK STREAM è TCP mentre SOCK_DGRAM è UDP
    if ( (socket_desc = socket(AF_INET, SOCK_STREAM, 0) ) == -1 ) //APERTURA SOCKET
        handle_error("errore creazione socket server_addr");

    server_addr.sin_addr.s_addr = INADDR_ANY; //compilo la struct del server,
    server_addr.sin_family = AF_INET; //non serve compilare il client
    server_addr.sin_port = htons(SERVER_PORT);
    if (bind(socket_desc, (struct sockaddr*)&server_addr, sockaddr_len)==-1) //BIND
        handle_error("errore binding server addr");
    ret = listen(socket_desc, MAX_CONN_QUEUE); //ASCOLTO
    if (ret < 0)
        handle_error("Cannot listen on socket");
    // loop to handle incoming connections (sequentially)
    while (1) {
        //ACCETTO LA RICHIESTA DI CONNESSIONE DAL CLIENT
        client_desc = accept(socket_desc, (struct sockaddr*)&client_addr, (socklen_t*)&sockaddr_len);
        if (client_desc < 0) handle_error("Cannot open socket for incoming connection");

        connection_handler(client_desc); //è una funzione che ci creiamo noi per gestire l'invio e la ricezione
    }
    exit(EXIT_SUCCESS); // this will never be executed
}

// Method for processing incoming requests. The method takes as argument
// the socket descriptor for the incoming connection.
void* connection_handler(int socket_desc) {
    int ret, recv_bytes, bytes_sent;

    char buf[1024];
    size_t buf_len = sizeof(buf);
    int msg_len;
    memset(buf,0,buf_len);

    char* quit_command = SERVER_COMMAND;
    size_t quit_command_len = strlen(quit_command);

    // send welcome message
    sprintf(buf, "Hi! I'm an echo server. I will send you back whatever"
        " you send me. I will stop if you send me %s", quit_command);
    msg_len = strlen(buf);
```

```

bytes_sent=0;
while ( bytes_sent < msg_len) {
    ret = send(socket_desc, buf + bytes_sent, msg_len - bytes_sent, 0); //INVIO //il numero di byte da
                                                                    //inviare. GUARDA A FINE PAGINA!

    if (ret == -1 && errno == EINTR) continue;
    if (ret == -1) handle_error("Cannot write to the socket");
    bytes_sent += ret;
}

if (DEBUG) fprintf(stderr, "Welcome message <<%s>> has been sent\n",buf);
// echo loop
while (1) {
    recv_bytes = 0;
    do {
        ret = recv(socket_desc, buf + recv_bytes, 1, 0); //RICEVO
        if (ret == -1 && errno == EINTR) continue;
        if (ret == -1) handle_error("Cannot read from the socket");
        if (ret == 0) break;
        } while ( buf[recv_bytes++] != '\n' );

    if (DEBUG) fprintf(stderr, "Received command of %d bytes...\n",recv_bytes);
    // check if either I have just been told to quit...
        if (recv_bytes == quit_command_len && !memcmp(buf, quit_command, quit_command_len)){
            break;
        }
    // ...or I have to send the message back
    bytes_sent=0;
    while ( bytes_sent < recv_bytes) {
        ret = send(socket_desc, buf + bytes_sent, recv_bytes - bytes_sent, 0); //INVIO conoscendo
                                                                    //il numero di byte
                                                                    //da inviare
        if (ret == -1 && errno == EINTR) continue;
        if (ret == -1) handle_error("Cannot write to the socket");
        bytes_sent += ret;
    }
}
ret = close(socket_desc); //CHIUDO
if (ret < 0) handle_error("Cannot close socket for incoming connection");
return NULL;
}

```

GUARDA QUI PER LA LETTURA DI UN NUMERO DI BYTE PER VOLTA FINO AL CARATTERE \n

```

//LEGGE UN CARATTERE PER VOLTA
do{
    ret = recv(socket_desc, buf+bytes_read, 1, 0);
    if (ret ==-1){
        if (errno == EINTR) continue;
        handle_error("errore nella read. probabile chiusura dei stream");
    }
    if (ret==0) break;
    }while (buf[bytes_read++]!='\n'); //ATTENZIONE! NOTA IL buf[bytes_read++] CON IL
++ DOPO! DEVE PRIMA LEGGERE IL buf[bytes_read] e poi incrementare altrimenti non troverà
mai il carattere \n

```

PER PATRIZIO CHE ODIA IL DO-WHILE

```
bytes_read = 0;
while(buf[bytes_read - 1] != '\n'){
    ret = recv(socket_desc, buf + bytes_read, 1, 0);
    if(ret == -1 && errno != EINTR){
        handle_error("[server] connection_handler recv error\n");
    }
    bytes_read += ret;
}
```

PATTERN PER RENDERE IL SERVER MULTIPROCESS OPPURE MULTITHREAD

// SE IL SERVER È MULTIPROCESS, ALLORA BISOGNA FARE QUESTO CICLO IN FASE DI ACCETTAZIONE

```
while (1) {
    int client = accept(server, .....);
    <gestione errori>
    pid_t pid = fork();
    if (pid == -1) {
        <gestione errori>
    } else if (pid == 0) {
        close(server);
        <elaborazione connessione client>
        _exit(0);
    } else {
        close(client);
    }
}
```

// SE IL SERVER È MULTIPROCESS, ALLORA BISOGNA FARE QUESTO CICLO IN FASE DI ACCETTAZIONE

//thread hanno un impatto minore alla CPU rispetto al multiprocess che ha un overhead più alto

```
while (1) {
    pthread_t thread;
    struct sockaddr_in* client_addr = calloc(1, sizeof(struct sockaddr_in)); //lo crea per ogni thread

    client_desc = accept(socket_desc, (struct sockaddr*) client_addr, (socklen_t*) &sockaddr_len);
    if (client_desc == -1 && errno == EINTR) continue; // check per segnali di interruzione
    if (client_desc == -1) handle_error("Impossibile aprire la socket per connessioni in ingre

    handler_args_t* arg = calloc(1, sizeof(handler_args_t));
    arg->socket_desc = client_desc;
    arg->client_addr = client_addr;
    arg->client_id = client_id;
    ret = pthread_create(&thread, NULL, connection_handler, arg);
    if(ret) handle_error_en(ret,"Errore nella creazione del thread");

    ret = pthread_detach(thread);
    if(ret) handle_error_en(ret,"Errore nel detach del thread");

    // incremento il client ID dopo ogni connessione accettata
    client_id++;
}
```


CLIENT

```
int main(int argc, char* argv[]) {
    int ret, bytes_sent, recv_bytes;

    // variables for handling a socket
    int socket_desc;

    struct sockaddr_in server_addr = {0}           //CREO STRUCT CLIENT
                                                    //porta TCP con SOCK_STREAM
    socket_desc = socket(AF_INET, SOCK_STREAM, 0); //APRO LA SOCKET DEL CLIENT,
                                                    //Usare SOCK_DGRAM per UDP

    if (socket_desc < 0)
        handle_error("Could not create socket");

                                                    // inet_addr importante per il client, poichè
    server_addr.sin_addr.s_addr = inet_addr(SERVER_ADDRESS); //il server accetta tutte le interface ma il
    server_addr.sin_family = AF_INET;                    //client deve convertire la stringa in un IP
    server_addr.sin_port = htons(SERVER_PORT); // don't forget about network byte order!

    //MI CONNETTO AL SERVER, BLOCCANTE FICNHÈ NON MI ACCETTA IL SERVER
    ret = connect(socket_desc, (struct sockaddr*)&server_addr, sizeof(struct sockaddr_in));
    if (ret < 0)
        handle_error("Could not create connection");

    char buf[1024];
    size_t buf_len = sizeof(buf);
    int msg_len;
    memset(buf, 0, buf_len);
    recv_bytes = 0;
    do {
//se UDP recvfrom(socket_desc, buf, buf_len, 0, NULL, NULL)
        ret = recv(socket_desc, buf + recv_bytes, buf_len - recv_bytes, 0); //RICEVO MSG SERVER
        if (ret == -1 && errno == EINTR) continue;
        if (ret == -1) handle_error("Cannot read from the socket");
        if (ret == 0) break;
        recv_bytes += ret;
    } while ( buf[recv_bytes-1] != '\n' ); //il -1 è importante perché sennò considero il carattere di fine stringa
//in C, cioè \0, e non troverà mai il \n che cerchiamo

    // main loop
    while (1) {
        char* quit_command = SERVER_COMMAND;
        size_t quit_command_len = strlen(quit_command);

        printf("Insert your message: ");

        /* Read a line from stdin
         *
         * fgets() reads up to sizeof(buf)-1 bytes and on success
         * returns the first argument passed to it. */
        if (fgets(buf, sizeof(buf), stdin) != (char*)buf) {
            fprintf(stderr, "Error while reading from stdin, exiting...\n");
            exit(EXIT_FAILURE);
        }
    }
}
```

```

    msg_len = strlen(buf);
//    buf[--msg_len] = '\0'; // remove '\n' from the end of the message

    bytes_sent=0;
    while ( bytes_sent < msg_len) {
//se è UDP, sendto(socket_desc, buf, msg_len, 0, (struct sockaddr*) &server_addr, sizeof(struct sockaddr_in))
        ret = send(socket_desc, buf + bytes_sent, msg_len - bytes_sent, 0);        //INVIO AL SERVER
        if (ret == -1 && errno == EINTR) continue;
        if (ret == -1) handle_error("Cannot write to the socket");
        bytes_sent += ret;
    }

    if (msg_len == quit_command_len && !memcmp(buf, quit_command, quit_command_len)){
        if (DEBUG) fprintf(stderr, "Sent QUIT command ...\n");
        break;
    }

    recv_bytes = 0;
    do {
//se UDP recvfrom(socket_desc, buf, buf_len, 0, NULL, NULL)
        ret = recv(socket_desc, buf + recv_bytes, buf_len - recv_bytes, 0);        //RICEVO (opzionale)
        if (ret == -1 && errno == EINTR) continue;
        if (ret == -1) handle_error("Cannot read from the socket");
            if (ret == 0) break;
            recv_bytes += ret;

    } while ( buf[recv_bytes-1] != '\n' );
}
ret = close(socket_desc);        //CHIUDO SOCKET CLIENT
if (ret < 0) handle_error("Cannot close the socket");

exit(EXIT_SUCCESS);
}

```

Dijkstra

Initially

```
/* global info */
boolean interested[N] = {false, ..., false};
boolean passed[N] = {false, ..., false};
int k = <any> ; // k ∈ {0, 1, ..., N-1}
/* local info */
int i = <entity ID>; // i ∈ {0, 1, ..., N-1}
```

repeat

```
1 <Non Critical Section>;
2 interested[i] = true;
3 while (k != i) {
4     passed[i] = false;
5     if (!interested[k]) then k = i;
6 }
7 passed[i] = true;
8 for j in 0 ... N-1 except i do
9     if (passed[j]) then goto 3;
10 <critical section>;
11 passed[i] = false; interested[i] = false;
```

forever

Possibili domande:

- cosa cambia se si nega la condizione a riga 9? Cioè 9
if (!passed[j]) then goto 3;
Quello che succede è che il j-esimo thread può essere passato ed aver accesso alla Critical Section, e la condizione passed[j]==True, allora la condizione !passed[j] == false, ed il thread i-esimo che ha effettuato questo check procede senza ritornare in cima al ciclo di partenza, ed allora non vi è più mutua esclusione
- Cosa cambia se nella riga 5 si sostituisce :
“if (!interested[k]) then k=i” con “if (interested[k]) then k=i”? c’è solo starvation perché il primo ciclo while, il k-esimo processo passa, ma nel frattempo che arriva al secondo ciclo for, potrebbe passare qualcun altro, facendo sì che il k-esimo ritorni al primo ciclo ed inoltre che k assume un nuovo valore. Inoltre il nuovo k, anche quando ricicla prima di effettuare il check di riga 3, può essere che un altro processo sovrascriva k, rispingendolo dentro al ciclo while. A fortuna un processo potrebbe sperare che esce dal primo ciclo while e nel frattempo i restanti processi rimangono chiusi nel primo ciclo, permettendogli di superare il secondo ciclo for ed entrare nella CS. Questo però è puramente casuale e provoca una forte Starvation.

Baker's Algorithm

Initially

/* global info */

boolean choosing[N] = {false, ..., false};

integer num[N] = {0, ..., 0};

/* local info */

int i = <entity ID>; // $i \in \{0, 1, \dots, N-1\}$

repeat

1 NCS

2 choosing[i] := true

3 $\text{num}[i] := 1 + \max \{ \text{num}[j] : 0 \leq j < N \}$

4 choosing[i] := false

5 for j := 0 to N-1 do begin

6 while choosing[j];

7 while $\text{num}[j] \neq 0$ AND $\{ \text{num}[j], j \} < \{ \text{num}[i], i \}$;

8 end

9 CS

10 $\text{num}[i] := 0$

Forever

Il $\text{num}[i]=0$ indica la volontà di non entrare nella critical section.

Possibili domande:

- Cosa cambia se in riga 3 non si somma il massimo dei numeri assegnati ai thread precedenti con l'1? Cioè $\text{num}[i] := \max \{ \text{num}[j] : 0 \leq j < N \}$ allora tutti avranno il numeretto pari a 0 a prescindere e la condizione $\text{num}[j] \neq 0$ in riga 7 è sempre false poiché $\text{num}[j] = 0$, ed allora poiché la condizione per rimanere nel ciclo non è soddisfatta, tutti i thread escono immediatamente ed addio mutua esclusione
- Cosa cambia se in riga 10 mettessi $\text{num}[i] = 1$? Allora il primo thread riesce pure a passare, ma si tiene il numero 1 finché non ritorna alla riga 3 a prendersi il numero finale nuovamente. Questo può provocare starvation in quanto finché il processo che è passato non si prende il numerino finale, allora i restanti processi attendono.
- Cosa succede se si inverte la riga 9 con la 10? Allora succede che non esiste più la mutua esclusione in quanto il processo che è uscito dal ciclo, prima di entrare dice che non ha volontà di entrare nella critical section, e quando ci entra prima che finisca permette ai restanti processi di uscire dal ciclo ed entrare insieme a lui nella CS.